2026-7-18 Linux进程实战--自主shell脚本

- 1. 自主 shell 脚本实战
  - 1.1 程序骨架与进程模型
    - 1.1.1 为什么要有一个永久循环
    - 1.1.2 一次命令的进程关系
  - 1.2 第一步：生成命令提示符
    - 1.2.1 提示符需要哪些信息
    - 1.2.2 提示符与目录显示
    - 1.2.3 提示符与进程状态必须同步
  - 1.3 第二步：读取一行命令
    - 1.3.1 为什么使用 fgets
    - 1.3.2 正确去除换行符
    - 1.3.3 空行和 EOF
  - 1.4 第三步：解析输入
    - 1.4.1 重定向与命令本体分开处理
    - 1.4.2 把命令行切成参数表
    - 1.4.3 解析结果决定后面的执行路径
  - 1.5 第四步：处理内建命令
    - 1.5.1 哪些命令不能放到子进程
    - 1.5.2 cd 与 PWD
    - 1.5.3 echo $? 与最近退出码
  - 1.6 第五步：执行外部命令
    - 1.6.1 子进程先设置环境，再调用 exec
    - 1.6.2 在子进程中完成重定向
    - 1.6.3 父进程等待并记录结果
    - 1.6.4 shell 也可以启动另一个 shell
  - 1.7 主循环如何连接各步骤
  - 1.8 编译、运行与当前实现边界
    - 1.8.1 使用 Makefile 编译
    - 1.8.2 可以验证的命令
    - 1.8.3 当前仓库代码需要继续完善的地方
    - 1.8.4 建议的改进顺序
  - 1.9 小结

1. 自主 shell 脚本实战

本笔记解析一个可以编译运行的小型命令行解释器，重点不是实现完整的 Bash，而是把进程控制知识串成可工作的程序：

> 打印提示符 → 读取命令行 → 解析命令 → 执行内建命令或外部程序 → 等待子进程 → 记录退出状态 → 回到循环。

当前目录中的主要文件是：

文件	作用
`myshell.cc`	shell 主程序
`makefile`	编译与清理规则
`log.txt`	测试时可直接使用的文本文件

shell 本身是一个长期运行的进程。外部命令不能直接替换 shell，否则命令结束后 shell 也消失了；因此外部命令采用 `fork + exec + wait`，而 `cd`、`echo $?` 等必须修改 shell 自身状态的命令则作为内建命令。

1.1 程序骨架与进程模型

1.1.1 为什么要有一个永久循环

真正执行完一条命令以后，shell 还要继续等待下一条命令。因此主逻辑必须放在循环中：

```
while (true)
{
    显示提示符
    读取一行输入
    解析重定向和命令参数

    如果是内建命令
        由 shell 自己执行
    否则
        fork 子进程
        子进程设置重定向并 exec
        父进程 waitpid

    保存上一条命令的退出状态
}
```

这个循环把多种命令统一成了相同的输入形式，但执行者不同：

命令类型	执行者	原因
`cd`	shell 自身	必须修改 shell 的工作目录
`echo $?`	shell 自身	必须读取 shell 保存的上次退出状态
`ls`、`cat` 等	子进程	新程序可以替换子进程，不应替换 shell
再启动一个 `./myshell`	子进程	shell 也可以作为普通程序被另一个 shell 启动

1.1.2 一次命令的进程关系

运行外部命令时，父子关系如下：

```
myshell
  │
  ├─ fork() ──→ 子进程
  │               │
  │               ├─ 设置 stdin/stdout 重定向
  │               └─ execvpe() 加载命令
  │
  └─ waitpid() ←── 命令退出
  │
  └─ 记录退出码，继续显示提示符
```

shell 负责创建命令进程和回收结果，子进程负责运行具体命令。了解这个分工后，剩余工作主要集中在输入处理、命令解析和状态维护上。

1.2 第一步：生成命令提示符

1.2.1 提示符需要哪些信息

常见的提示符格式是：

```
[用户名@主机名 当前目录]#
```

项目为了突出进程控制，直接从环境变量中读取 `USER`、`HOSTNAME` 和 `PWD`：

```c
const char *GetUserName()
{
    char *name = getenv("USER");
    return name == NULL ? "None" : name;
}

const char *GetHostName()
{
    char *hostname = getenv("HOSTNAME");
    return hostname == NULL ? "None" : hostname;
}

const char *GetPwd()
{
    char *pwd = getenv("PWD");
    return pwd == NULL ? "None" : pwd;
}
```

这种写法简单，适合先完成 shell 主流程。环境变量不存在时不能直接解引用空指针，所以每个函数都提供了 `"None"` 作为回退值。

系统级接口还可以提供更直接的信息来源：

```
getuid(2)

NAME
       getuid, geteuid - get user identity

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <unistd.h>

       uid_t getuid(void);
       uid_t geteuid(void);
```

`getuid()` 返回当前用户 ID，而不是用户名。如果需要用户名，一般还要配合 `getpwuid()` 查询用户数据库。

```
NAME
       gethostname, sethostname - get/set hostname

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <unistd.h>

       int gethostname(char *name, size_t len);
       int sethostname(const char *name, size_t len);
```

`gethostname()` 直接读取本机主机名，比依赖某个环境变量更稳定。

```
getcwd(3)

NAME
       getcwd, getwd, get_current_dir_name - get current working directory

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <unistd.h>

       char *getcwd(char buf[.size], size_t size);
       char *get_current_dir_name(void);
```

`getcwd()` 直接询问内核当前工作目录。环境变量 `PWD` 与真实工作目录不一定是同一状态，尤其在目录被移动、程序主动 `chdir()` 或环境变量未同步时。

1.2.2 提示符与目录显示

项目用 `rfindDir()` 只保留当前路径的最后一个目录：

```cpp
static std::string rfindDir(const std::string &path)
{
    if (path == "/")
        return path;

    auto pos = path.rfind('/');
    if (pos == std::string::npos)
        return std::string();

    return path.substr(pos + 1);
}
```

例如：

```
/home/whb/project -> project
/home             -> home
/                 -> /
```

提示符打印函数需要主动刷新标准输出：

```
void PrintCommandLine()
{
    printf("[%s@%s %s]# ",
           GetUserName(),
           GetHostName(),
           rfindDir(GetPwd()).c_str());

    fflush(stdout);
}
```

提示符末尾没有换行，`fflush(stdout)` 可以确保它立即显示，而不是留在缓冲区中等待下一次刷新。

1.2.3 提示符与进程状态必须同步

shell 切换目录后，不只是当前工作目录改变，供后续提示符使用的路径信息也要同步更新。

项目在 `cd` 内建命令中同时做了两件事：

1. 调用 `chdir()` 修改 shell 进程的真实工作目录；
2. 重新取得当前路径，并通过 `putenv()` 更新 `PWD`。

如果只改其中一项，就容易出现“实际目录已经变了，但提示符仍显示旧路径”的现象。

1.3 第二步：读取一行命令

1.3.1 为什么使用 `fgets`

`scanf("%s", ...)` 会在空白处停止，不能一次读取完整命令行。`fgets()` 可以读取包括空格在内的一整行：

```
NAME
       fgetc, fgets, getc, getchar, ungetc - input of characters and strings

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <stdio.h>

       int fgetc(FILE *stream);
       int getc(FILE *stream);
       int getchar(void);
```

项目使用固定大小缓冲区：

```
char command_line[MAXSIZE] = {0};

if (fgets(command_line, sizeof(command_line), stdin) == NULL) {
    /* 读取失败或遇到 EOF */
}
```

选择 `fgets` 时还要处理三个细节：

- 它会把换行符一并读入；
- 输入超过缓冲区时可能只读到前半段，剩余内容仍留在输入流中；
- 到达 EOF 或读取失败时返回 `NULL`。

1.3.2 正确去除换行符

项目当前代码使用：

```
if (fgets(commandline, size, stdin) == NULL)
    return 0;

commandline[strlen(commandline) - 1] = '\0';
return strlen(commandline);
```

这段代码在一个正常短行中可以工作，但没有判断最后一个字符是否真的为 `\n`。当缓冲区正好填满、或者输入来自文件且最后一行没有换行时，它会误删有效字符。

更稳妥的写法是使用 `strcspn()`：

```
if (fgets(commandline, size, stdin) == NULL)
    return -1;

commandline[strcspn(commandline, "\n")] = '\0';
return strlen(commandline);
```

这里把 EOF/读失败与“读到空行”区分开，方便主循环在 EOF 时真正退出。

如果输入超过缓冲区，还应继续读取并丢弃该行剩余内容，避免下一次循环把半截命令当成新命令。

1.3.3 空行和 EOF

主循环至少应区分三种结果：

情况	应做的处理
读取到普通命令	继续解析
读取到空行或全空白行	直接进入下一轮
EOF 或读取失败	结束 shell

如果对空行继续执行 `ParseCommand()`，`gargv[0]` 会变成空指针。随后 `CheckBuiltinExecute()` 调用 `strcmp(gargv[0], "cd")` 时就会解引用空指针。

因此，解析之前必须先判断有效命令是否存在。

1.4 第三步：解析输入

1.4.1 重定向与命令本体分开处理

项目在切分参数之前先处理重定向：

```
ls -a -l > out.txt
```

目标是把原字符串拆成两部分：

```
命令部分：ls -a -l
重定向：输出到 out.txt
```

`myshell.cc` 用四个状态记录重定向方式：

```
#define NoneRedir   0
#define InputRedir  1
#define AppRedir    2
#define OutputRedir 3
```

它们分别表示：

- `NoneRedir`：没有重定向；
- `InputRedir`：`<`，标准输入重定向；
- `OutputRedir`：`>`，覆盖写入；
- `AppRedir`：`>>`，追加写入。

`ParseRedir()` 从左到右扫描命令，遇到重定向符号时在原字符串中写入 `\0`：

```
"ls -a -l\0> out.txt"
```

这样前半段仍可交给 `ParseCommand()`，`filename` 指向后半段的文件名。这是一个原位修改字符串的解析技巧。

当前实现只处理第一个重定向符号，不支持引号、多个重定向和管道。对于教学版 shell，够用；对于复杂命令，仍应使用真正的词法分析。

1.4.2 把命令行切成参数表

命令字符串例如：

```
ls -a -l
```

需要转换成：

```
gargv[0] = "ls"
gargv[1] = "-a"
gargv[2] = "-l"
gargv[3] = NULL
```

项目使用 `strtok()`：

```
NAME
       strtok, strtok_r - extract tokens from strings

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <string.h>

       char *strtok(char *restrict str, const char *restrict delim);
       char *strtok_r(char *restrict str, const char *restrict delim,
                      char **restrict saveptr);
```

```
ZL@VM-0-14-ubuntu:~/code/lesson12-shell$ ./myshell
[ZL@None /home/ZL/code/lesson12-shell]# ls -a -l
gargc: 3
gargv[0]: ls
gargv[1]: -a
gargv[2]: -l
```

核心代码如下：

```
gargc = 0;
memset(gargv, 0, sizeof(gargv));

gargv[0] = strtok(commandline, " ");
while (gargv[++gargc] = strtok(NULL, " ")) {
    /* strtok 已经把分隔符改写成 \0 */
}
```

`strtok()` 会把分隔符替换成字符串结束符，所以切分后的每一项都能单独作为 C 字符串交给 `exec`。

这段代码需要补充边界检查，避免参数超过 `MAXARGS`：

```
gargc = 0;
memset(gargv, 0, sizeof(gargv));

gargv[gargc] = strtok(commandline, " \t");
while (gargv[gargc] != NULL && gargc < MAXARGS - 1) {
    gargv[++gargc] = strtok(NULL, " \t");
}
```

最后一个位置必须留给 `NULL`，因为 `exec` 的参数数组依赖它判断结束。

1.4.3 解析结果决定后面的执行路径

解析完成后，程序已经知道：

- 程序名是 `gargv[0]`；
- 参数数组是 `gargv`；
- 参数个数是 `gargc`；
- 是否存在重定向，以及目标文件名是什么。

下面只需要回答一个问题：这条命令由 shell 自己执行，还是交给子进程执行。

1.5 第四步：处理内建命令

1.5.1 哪些命令不能放到子进程

`cd` 的典型错误实现是：

```
pid_t id = fork();
if (id == 0) {
    chdir(argv[1]);
    exit(0);
}
```

这样改变的只是子进程目录。子进程退出后，父 shell 的工作目录没有变化。

内建命令必须在 shell 自身进程内执行。项目中的判断流程如下：

```
[ZL@None /home/ZL/code/lesson12-shell]# pwd
/home/ZL/code/lesson12-shell
[ZL@None /home/ZL/code/lesson12-shell]# cd ../
[ZL@None /home/ZL/code/lesson12-shell]# pwd
/home/ZL/code
[ZL@None /home/ZL/code/lesson12-shell]#
```

```
if (CheckBuiltinExecute()) {
    continue;
}

ExecuteCommand();
```

`CheckBuiltinExecute()` 返回非零表示命令已由 shell 处理，本轮不应再创建子进程。

1.5.2 `cd` 与 `PWD`

`chdir()` 是切换当前进程工作目录的系统调用：

```
NAME
       chdir, fchdir - change working directory

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <unistd.h>

       int chdir(const char *path);
       int fchdir(int fd);
```

项目中的主要逻辑是：

```
if (strcmp(gargv[0], "cd") == 0) {
    if (gargc == 2) {
        if (chdir(gargv[1]) == 0) {
            char pwd[1024];

            if (getcwd(pwd, sizeof(pwd)) != NULL) {
                snprintf(cwd, sizeof(cwd), "PWD=%s", pwd);
```

```
                putenv(cwd);
                lastcode = 0;
            }
        } else {
            perror("cd");
            lastcode = 1;
        }
    }
    return 1;
}
```

这里发生了两种状态变化：

1. 内核中该进程的当前工作目录改变；
2. shell 自己维护的 `PWD` 环境变量同步更新。

`snprintf()` 把格式化的 `PWD=...` 写入缓冲区：

```
int printf(const char *restrict format, ...);
int fprintf(FILE *restrict stream,
            const char *restrict format, ...);
int dprintf(int fd,
            const char *restrict format, ...);
int sprintf(char *restrict str,
            const char *restrict format, ...);
int snprintf(char str[restrict .size], size_t size,
            const char *restrict format, ...);

int vprintf(const char *restrict format, va_list ap);
int vfprintf(FILE *restrict stream,
            const char *restrict format, va_list ap);
int vdprintf(int fd,
            const char *restrict format, va_list ap);
int vsprintf(char *restrict str,
            const char *restrict format, va_list ap);
int vsnprintf(char str[restrict .size], size_t size,
            const char *restrict format, va_list ap);
```

它与 `printf()` 的格式规则相近，区别是输出写到指定缓冲区，而不是显示到终端。

`putenv(cwd)` 只是把字符串地址交给环境变量表，不会复制字符串。因此 `cwd` 必须是生命周期足够长的存储，项目使用全局数组是合理的。

1.5.3 `echo $?` 与最近退出码

shell 需要保存最近一条命令是否成功，才能支持：

```
echo $?
```

项目中的 `lastcode` 保存最近一次外部命令或关键内建命令的退出码。

正常结束的命令可读取退出码：

```
lastcode = WEXITSTATUS(status);
```

但只在 `WIFEXITED(status)` 为真时才能使用这个宏。若命令被信号终止，应记录信号：

```
if (WIFEXITED(status)) {
    lastcode = WEXITSTATUS(status);
} else if (WIFSIGNALED(status)) {
    lastcode = 128 + WTERMSIG(status);
}
```

项目当前版本的 `echo` 内建只处理 `$?` 和 `$PATH`。普通的 `echo hello` 还需要补充直接输出参数的逻辑，否则它可能被误判为“已处理”却没有输出。

1.6 第五步：执行外部命令

1.6.1 子进程先设置环境，再调用 `exec`

`ExecuteCommand()` 创建子进程：

```
pid_t id = fork();
if (id < 0) {
    perror("fork");
    return -1;
}

if (id == 0) {
    /* 子进程处理重定向，然后 exec */
}
```

如果子进程直接自行处理命令，那么它仍然要理解所有命令语义；更好的方式是设置好运行环境后，让 `execvpe()` 加载真正的程序。

项目使用：

```
execvpe(gargv[0], gargv, genv);
perror("execvpe");
_exit(127);
```

选择 `execvpe()` 的原因是：

- `v`：参数已经整理为 `gargv` 数组；
- `p`：只写 `ls` 这类命令名时，可以搜索 `PATH`；
- `e`：新程序使用显式环境变量表 `genv`。

`execvpe()` 是 GNU/Linux 扩展。如果只需要继承 shell 当前环境，使用标准接口 `execvp(gargv[0], gargv)` 更简单。

1.6.2 在子进程中完成重定向

打开文件和复制文件描述符的操作放在子进程中完成，因为重定向只应影响这一条命令，不能永久改变父 shell 的标准输入输出。

覆盖输出：

```
int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0666);
dup2(fd, STDOUT_FILENO);
```

追加输出：

```
int fd = open(filename, O_WRONLY | O_CREAT | O_APPEND, 0666);
dup2(fd, STDOUT_FILENO);
```

输入重定向：

```
int fd = open(filename, O_RDONLY);
```

```
dup2(fd, STDIN_FILENO);
```

`dup2(fd, target)` 会让目标文件描述符指向同一个打开文件，因此：

```
dup2(fd, 1) -> 后续标准输出写入文件
dup2(fd, 0) -> 后续标准输入从文件读取
```

当前代码没有检查 `open()` 和 `dup2()` 是否失败。更完整的实现应在失败时执行 `perror()` 和 `_exit(1)`，避免把一个无效文件描述符交给新程序。

1.6.3 父进程等待并记录结果

子进程执行命令时，父 shell 继续运行自己的等待逻辑：

```
int status = 0;
pid_t rid = waitpid(id, &status, 0);

if (rid == id) {
    if (WIFEXITED(status)) {
        lastcode = WEXITSTATUS(status);
    } else if (WIFSIGNALED(status)) {
        lastcode = 128 + WTERMSIG(status);
    }
}
```

一次运行结果如下：

```
[ZL@None /home/ZL/code/lesson12-shell]# ls -a -l --color
total 36
drwxrwxr-x  2 ZL ZL  4096 Jul 18 20:31 .
drwxrwxr-x 18 ZL ZL  4096 Jul 18 18:44 ..
-rw-rw-r--  1 ZL ZL    66 Jul 18 12:27 Makefile
-rwxrwxr-x  1 ZL ZL 16880 Jul 18 20:28 myshell
-rw-rw-r--  1 ZL ZL  3058 Jul 18 20:31 myshell.c
[ZL@None /home/ZL/code/lesson12-shell]# top
top - 20:32:31 up 14 days, 21:17,  6 users,  load average: 0.00, 0.00, 0.00
Tasks: 138 total,   1 running, 135 sleeping,   2 stopped,   0 zombie
%Cpu(s):  0.7 us,  0.3 sy,  0.0 ni, 99.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   1967.6 total,    130.8 free,    543.1 used,   1469.9 buff/cache
MiB Swap:   1988.0 total,   1987.5 free,      0.5 used.   1424.5 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   2368 root      20   0  778844  29100   6156 S   1.3   1.4 217:58.67 barad_agent
 164011 root      20   0 1793288  47060  33152 S   0.7   2.3  17:43.43 containerd
3864123 root      20   0 1031288  90108  34852 S   0.7   4.5  37:41.01 YDService
 719686 root      20   0   14088   9752   8336 S   0.3   0.5   0:00.01 sshd
```

父进程等待的意义不仅是“让命令先执行完”，还包括：

- 回收子进程，避免僵尸进程；
- 取得子进程是否正常退出；
- 保存退出码，供后面的 `echo $?` 使用。

1.6.4 shell 也可以启动另一个 shell

外部命令执行机制并不只针对系统命令。`./myshell` 本身也是可执行程序，因此第一条 shell 可以通过 `fork + exec` 启动第二条 shell：

```
ZL@VM-0-14-ubuntu:~/code/lesson12-shell$ ps axj | grep shell
 708100  708216  708216  708100 pts/2     708216 Sl+   1002   0:01 vim myshell.c
 722063  722365  722365  722063 pts/5     722365 S+    1002   0:00 ./myshell
 722365  722883  722365  722063 pts/5     722365 S+    1002   0:00 ./myshell
 722883  723004  722365  722063 pts/5     722365 S+    1002   0:00 ./myshell
 717081  723019  723018  717081 pts/7     723018 S+    1002   0:00 grep --color=auto shell
```

这体现了 shell 的两个身份：

1. 对自己是命令解释器；
2. 对其他 shell 或程序来说，只是一个普通的命令行程序。

1.7 主循环如何连接各步骤

按照项目要实现的功能，并在补齐 EOF、空行和参数表边界处理后，可以把主循环整理为：

```c
int main()
{
    char command_line[MAXSIZE] = {0};

    while (true) {
        PrintCommandLine();

        int len = GetCommand(command_line, sizeof(command_line));
        if (len < 0) {
            break;              /* EOF，退出 shell */
        }
        if (len == 0) {
            continue;           /* 空行，不解析 */
        }

        ParseRedir(command_line);
```

```
        if (ParseCommand(command_line) == 0) {
            continue;
        }

        if (CheckBuiltinExecute()) {
            continue;
        }

        ExecuteCommand();
    }

    return 0;
}
```

各函数职责如下：

函数	职责
`PrintCommandLine()`	打印用户名、主机名和目录
`GetCommand()`	读取一行并去除换行
`ParseRedir()`	识别 `<`、`>`、`>>`
`ParseCommand()`	使用 `strtok()` 构造 `gargv`
`CheckBuiltinExecute()`	处理必须由 shell 自身执行的命令
`ExecuteCommand()`	`fork + exec + wait` 执行外部命令

主循环中每一步只负责一种任务，调试时也更容易判断问题发生在读取、解析、内建命令还是外部程序阶段。

1.8 编译、运行与当前实现边界

1.8.1 使用 Makefile 编译

仓库中的 Makefile 很简单：

```
myshell:myshell.cc
    g++ -o $@ $^ -g -std=c++11

.PHONY: clean
```

```
clean:
    rm -f myshell
```

在当前目录执行：

```
make
./myshell
make clean
```

编译和运行应在 Linux 环境中进行，因为程序依赖 `fork()`、`execvpe()`、`waitpid()`、`chdir()` 等 POSIX/Linux 接口。

1.8.2 可以验证的命令

可以按功能逐项测试：

```
ls -a -l
pwd
cd ..
pwd
echo $?
ls -a -l > log.txt
cat < log.txt
./myshell
```

测试时重点观察：

- 提示符是否显示当前用户、主机名和目录；
- `cd` 后提示符与实际路径是否同时变化；
- 外部命令执行后 shell 是否仍能继续接受输入；
- `echo $?` 是否能取得上一次命令状态；
- 重定向是否只影响子进程，不影响下一条命令。

1.8.3 当前仓库代码需要继续完善的地方

这份源码适合作为“自主 shell 的完整骨架”，但还不是生产级 shell。阅读时尤其要注意：

1. `LoadEnv()` 当前在主函数中被注释，导致 `genv` 只包含结尾的 `NULL`。如果要用 `execvpe(..., genv)`，应先正确加载环境；若只想继承当前环境，改用 `execvp()`。
2. `LoadEnv()` 使用固定 32 项数组和固定 4096 字节缓冲区，环境变量数量或长度超过上限时存在越界风险。
3. `GetCommand()` 用 `commandline[strlen(...) - 1]` 删除换行，无法正确处理最后一行没有换行或空字符串的情况。
4. 空行和全空白输入在解析后可能产生 `gargv[0] == NULL`，直接进入内建命令判断会崩溃。
5. `fgets()` 遇到 EOF 后，当前主循环会继续打印提示符，应该用明确的返回码结束循环。
6. `ParseCommand()` 没有限制参数数量，超过 `MAXARGS` 时可能越界。
7. `open()`、`dup2()` 的返回值没有检查，重定向失败时仍可能继续调用 `execvpe()`。
8. 父进程没有先使用 `WIFEXITED()` 判断就直接调用 `WEXITSTATUS()`；异常终止时退出码没有业务意义。
9. `execvpe()` 失败后使用 `exit(1)`，在 `fork` 场景中更建议使用 `_exit(127)`，避免重复刷新继承的缓冲区。
10. `CheckBuiltinExecute()` 只实现了 `cd`、`echo $?` 和 `echo $PATH` 的骨架，普通 `echo 参数`、`export`、环境变量展开仍未完整实现。
11. 当前不支持引号、转义、管道、命令组合、后台任务和作业控制，这些都属于后续扩展，不属于这套最小实现必须立即完成的部分。

1.8.4 建议的改进顺序

先保证基础骨架正确，再逐步增加功能：

1. 处理 EOF、空行、超长输入和参数表越界；
2. 检查 `fork()`、`open()`、`dup2()`、`exec` 和 `waitpid()` 的返回值；
3. 统一退出状态的记录方式；
4. 完善内建命令和 `PWD` 更新；
5. 最后再扩展引号、管道、重定向组合和作业控制。

这样的顺序能保证每一步都有明确的进程模型，避免功能增加后难以定位问题。

1.9 小结

这套 shell 的主线可以压缩成六个动作：

1. 用循环持续显示提示符；
2. 用 `fgets()` 读取完整命令行；

3. 先解析重定向，再用 `strtok()` 构造 `gargv`；
4. `cd`、`echo $?` 等命令由 shell 自己执行；
5. 其他命令由子进程通过 `fork + exec` 执行；
6. 父进程通过 `waitpid()` 回收子进程并记录状态。

真正需要理解的重点不是记住每一个函数，而是分清“谁应该执行”：修改 shell 状态的命令必须留在 shell 进程中，需要独立运行的外部程序则交给子进程；父进程始终负责等待、回收和继续循环。